User's Guide

TBtrans 5.2.2

February 04, 2025

https://gitlab.com/siesta-project/siesta

Contributors to TBtrans

 The original TBtrans code was implemented by Mads Brandbyge, Jose L. Mozos, Jeremy Taylor, Pablo Ordejon and Kurt Stokbro. The current TBtrans is implemented by the following contributors:

Nick Rübner Papior *Technical University of Denmark*

Contents

1 Introduction

This Reference Manual contains descriptions of all the input, output and execution features of TBTRANS, but is not a tutorial introduction to the program.

TBTRANS (Tight-Binding transport) is a generic computer program which calculates transport and other physical quantities using the Green function formalism. It is a stand-alone program which allows *extreme* scale tight-binding calculations.

- It uses the basic non-equilibrium Green function formalism and allows extensive customizability and analysis forms.
- TBTRANS may be given any type of local-orbital Hamiltonian and calculate transport properties of arbitrary geometries and/or number of electrodes.
- The PHTRANS variant may be compiled to obtain thermal (phonon) transport using the same Green function formalism and *all* the same functionalities as those presented in this manual.

As TBTRANS output has changed to the flexible NetCDF-4 format you are highly encouraged to use the SISL[3] toolsuite which has nearly all the necessary tools available to perform advanced analysis. If used, please cite SISL appropriately.

A list of the currently implemented features are:

- Density of states (orbital resolved)
  - Green function DOS
  - Scattering DOS
- Hamiltonian interpolation at different voltages
- Selective wide-band limit of the electrode(s)
- Transmission eigenvalues
- Bulk electrode density of state and transmission (directly from the electrode Hamiltonian)
- Projected transmission of eigenstates
- Orbital resolved "bond-currents" which may subsequently be analyzed to yield actual bond-currents
- Density matrices using the Green function and/or the spectral density
- COOP and COHP curves using the Green function and/or the spectral density.

References:

- Description of the TBTRANS and TRANSIESTA code in the N terminal generic implementation[1].
- SISL is a data analysis/extraction utility for TBTRANS which enables easy access to the data stored in the output NetCDF-4 file[3].

1.1 PHtrans

The NEGF formalism also applies to phonons via some simple differences. Here is a list of some of the differences:

- For PHtrans all options are *still* prefixed with **TBT**!
- The Green function calculation looks like:

$$\mathbf{G}_{\mathbf{q}} = [(\omega^2 + i\eta^2)\mathbf{I} - \mathbf{D}_{\mathbf{q}} - \mathbf{\Sigma}_{\mathbf{q}}(\omega)]^{-1}, \tag{1}$$

  where ω is referred to as *energy* in the remaining document.
- Calculating density matrices (**TBT.DM.Gf**, **TBT.COOP.Gf**, **TBT.COHP.Gf**) are prefactored with 2ω which is currently empirically done.

NOTE: PHtrans is not as tested as TBtrans. Any feedback on all parts are most welcome!

2 Compilation

TBtrans will be compiled when together with SIESTA when issuing `cmake`.

If only the TBtrans/PHtrans executables are desired, say for tight-binding calculations, one should execute the `cmake` build like this:

```
cmake -B _build --target SIESTA.tbtrans SIESTA.phtrans
```

3 Execution of the Program

TBtrans should be called with an input file which defines what *it should do*. This may either be piped or simply added on the input line. The latter method is preferred as one may use flags for the executable.

```
$ tbtrans < RUN.fdf
$ tbtrans RUN.fdf
```

Note that if TBtrans is compiled with MPI support one may call it like

```
$ mpirun -np 4 tbtrans RUN.fdf
```

for 4 MPI-processors.

TBtrans has these optional flags:

-help or **-h** print a help instruction and quit

-version or **-v** print TBtrans version and quit

-out or **-o** specify where all output should be written to (instead of STDOUT)

-L override **SystemLabel** flag

-V override **TBT.Voltage** flag. To denote the unit do as this example: **-V 0.2 eV** which sets the voltage to 0.2 eV. A value without unit is interpreted as eV.

-D override **TBT.Directory** flag, all output of TBtrans will be put in the corresponding folder (it will be created if non-existing)

-HS specify the **TBT.HS** variable, quickly override the used Hamiltonian

-fdf specify any given fdf flag on the command line, example **-fdf TBT.Voltage 0.2 eV**

Note that for all flags one may use ":" as a replacement for " ", although one may use quotation marks when having a space in the argument.

4 fdf-flags

Although TBtrans is a fully independent Green function transport code, it is hard-wired with the TranSIESTA fdf flags and options. If you are familiar with TranSIESTA and its input flags, then the use of TBtrans should be easy.

All fdf-flags for TBtrans are defaulted to their equivalent TranSIESTA flag. Thus if you are using TranSIESTA as a back-end you should generally not change any flags. For instance **TBT.Voltage** defaults to **TS.Voltage** if not supplied.

SystemLabel **siesta** *(string)*

The label defining this calculation. All relevant output will be prefixed with the **SystemLabel**.

One may start several TBtrans calculations in the same directory if they have different labels.

TBT.Voltage 0 eV *(energy)*

Define the applied bias in the scattering region.

TBT.Directory **./** *(directory)*

Define the output directory of files from TBtrans. This allow execution of several TBtrans instances in the same folder and writing their result to different, say, sub-folders. It is particularly useful for interpolation of Hamiltonian's and for testing purposes.

TBT.Verbosity **5** *(integer)*

Specify how much information TBtrans will print-out (range 0-10).

For smaller numbers, less information will be printed, and for larger values, more information is printed.

TBT.Progress **5.** *(real)*

TBtrans prints out an estimated time of completion (ETA) for the calculation. By default this is printed out every 5% of the total loops (k-point $\times$ energy loops). Setting this to 0 will print out after every energy loop.

4.1 Define electronic structure

TBT.HS <SystemLabel>.TSHS *(file)*

Define the Hamiltonian file which contains information regarding the Hamiltonian and geometry.

%block TBT.HS.Files ⟨None⟩ *(block)*

A list of files which each contain the Hamiltonian for the same geometry at different bias'. Each line has three entries, 1) the **TBT.HS** file, 2) the value of the bias applied, 3) the unit of the bias.

NOTE: if this is existing it will assume that you will perform an interpolation of the Hamiltonians to the corresponding bias (**TBT.Voltage**).

TBT.HS.Interp spline|linear *(string)*

depends on: **TBT.HS.Files**

Interpolate all files defined in **TBT.HS.Files** to the corresponding applied bias.

Generally **spline** produces the best interpolated values and its use is encouraged. The linear interpolation scheme is mainly used for comparison to the **spline**. If they are very different from each other then one may be required to perform additional self-consistent calculations at the specific bias due to large changes in the electronic structure.

Say you have calculated the SCF solution of a certain system at 5 different applied bias':

```
%block TBT.HS.Files
  ../V0/siesta.TSHS     0.  eV
  ../V-0.5/siesta.TSHS -0.5 eV
  ../V0.5/siesta.TSHS   0.5 eV
  ../V-1.0/siesta.TSHS -1.0 eV
  ../V1.0/siesta.TSHS   1.0 eV
%endblock
```

and you wish to calculate the interpolated transmissions and currents at steps of 0.1 eV, then you may use this simple loop

```
for V in `seq -1.5 0.1 1.5` ; do
   tbtrans -V $V:eV -D V$V RUN.fdf
done
```

which at each execution of TBtrans interpolates the Hamiltonian to the corresponding applied bias and store all output files in the `V$V` folder.

4.1.1 Changing the electronic structure via δ elements

The electronic structure may be altered by changing the Hamiltonian elements via a simple additive term

$$\mathbf{H} \leftarrow \mathbf{H} + \delta\mathbf{H} + \delta\mathbf{\Sigma}, \tag{2}$$

which allows easy changes to the electronic structure or adding additional terms such as imaginary self-energies. One may also use it to add magnetic fields etc.

TBtrans uses a distinction between $\delta\mathbf{H}$ and $\delta\mathbf{\Sigma}$ only via the orbital current calculation. I.e. $\delta\mathbf{H}$ enters the equations for calculating the orbital current, whereas $\delta\mathbf{\Sigma}$ does not. Otherwise the two

δ-terms are *completely* identical. In the following discussion we will use the term δ to be either $\delta\mathbf{H}$ or $\delta\mathbf{\Sigma}$.

To use this feature at k points it is important to know that phases in TBTRANS are defined using the lattice vectors (and *not* inter-atomic distances)

$$\mathbf{H}_k = \mathbf{H} \cdot e^{ik\cdot\mathbf{R}}. \tag{3}$$

TBTRANS will add the phases on *all* elements of δ via Eq. (3). To counter these phases one may simply multiply δ with the negative phase $(-i)$. Note that phases are only added on super cell elements, *not* unit cell elements.

TBT.dH ⟨**None**⟩ *(file)*

Denote a file which contains the $\delta\mathbf{H}$ information.

NOTE: that the terms defined in this file are added to the Hamiltonian when calculating the orbital currents, if your terms are not a Hamiltonian change, then consider using **TBT.dSE** instead.

This file *must* adhere to these file format notations and is required to be supplied in a NetCDF4 format

```
netcdf file.dH {
dimensions:
        one = 1 ;
        n_s = 9 ;
        xyz = 3 ;
        no_u = 900 ;
        spin = 1 ;
variables:
        int nsc(xyz) ;
                nsc:info = "Number of supercells in each unit-cell direction" ;

group: LEVEL-1 {
  dimensions:
          nnzs = 2670 ;
  variables:
          int n_col(no_u) ;
                  n_col:info = "Number of non-zero elements per row" ;
          int list_col(nnzs) ;
                  list_col:info = "Supercell column indices in the sparse format" ;
          int isc_off(n_s, xyz) ;
                  isc_off:info = "Index of supercell coordinates" ;
          double Redelta(spin, nnzs) ;
                  Redelta:info = "Real part of delta" ;
                  Redelta:unit = "Ry" ;
          double Imdelta(spin, nnzs) ;
                  Imdelta:info = "Imaginary part of delta" ;
                  Imdelta:unit = "Ry" ;
  } // group LEVEL-1

group: LEVEL-2 {
  dimensions:
          nkpt = UNLIMITED ;
          nnzs = 2670 ;
  variables:
          double kpt(nkpt, xyz) ;
                  kpt:info = "k-points for delta values" ;
```

```
                kpt:unit = "b**-1" ;
        ... n_col list_col isc_off ...
        double delta(nkpt, spin, nnzs) ;
                delta:info = "delta" ;
                delta:unit = "Ry" ;
  } // group LEVEL-2

group: LEVEL-3 {
  dimensions:
        ne = UNLIMITED ;
        nnzs = 2670 ;
  variables:
        double E(ne) ;
                E:info = "Energy points for delta values" ;
                E:unit = "Ry" ;
        ... n_col list_col isc_off ...
        double delta(ne, spin, nnzs) ;
                delta:info = "delta" ;
                delta:unit = "Ry" ;
  } // group LEVEL-3

group: LEVEL-4 {
  dimensions:
        nkpt = UNLIMITED ;
        ne = UNLIMITED ;
        nnzs = 2670 ;
  variables:
        double kpt(nkpt, xyz) ;
                kpt:info = "k-points for delta values" ;
                kpt:unit = "b**-1" ;
        double E(ne) ;
                E:info = "Energy points for delta values" ;
                E:unit = "Ry" ;
        ... n_col list_col isc_off ...
        double delta(nkpt, ne, spin, nnzs) ;
                delta:info = "delta" ;
                delta:unit = "Ry" ;
  } // group LEVEL-4
}
```

This example file shows how the file should be formatted. Note that one may either define the Hamiltonian as `delta` or as `Redelta` and `Imdelta`. The former is defining δ as a real quantity while the latter makes it an imaginary δ.

The levels are defined because they have precedence from each other, if the energy point and k point is found in LEVEL-4 it will use this, if not, it will check for the energy point in LEVEL-3, and so on.

The remaining options are only applicable if **TBT.dH** has been set.

TBT.dH.Parallel **true** *(logical)*

Whether the $\delta\mathbf{H}$ file should be read in parallel. If your architecture supports parallel IO it is beneficial to do so. TBtrans performs a basic check whether parallel IO may be possible, if it cannot assert this it will be turned off.

TBT.dSE ⟨**None**⟩ *(file)*

File has same format as specified for **TBT.dH**.

The only difference between a $\delta\mathbf{H}$ and $\delta\mathbf{\Sigma}$ file is that the terms in $\delta\mathbf{\Sigma}$ does *not* enter the calculation of the bond-currents, whereas $\delta\mathbf{H}$ does, see Eq. (16).

4.2 Determine calculated physical quantities

TBtrans can calculate a large variety of physical quantities. By default it will only calculate the transmission between the electrodes. Calculating as few quantities as possible will increase throughput, while requesting many quantities will result in much longer run-times.

You are heavily encouraged to compile TBtrans with NetCDF4 support, see Sec. **??**, as quantities will be orbital resolved.

If TBtrans has been compiled with NetCDF4 support, one may extract the projected DOS from the `SystemLabel.TBT.nc` using sisl (or manual scripting). The calculated DOS can *only* be extracted from the atoms in the device region (atoms in block **TBT.Atoms.Device**). Hence the **TBT.Atoms.Device** block is *extremely* important when conducting detailed DOS analysis. For instance if the input file has this:

```
%block TBT.Atoms.Device
  atom [20 -- 40]
%endblock
```

one may extract the PDOS on a subset of atoms using this sisl command

```
sdata siesta.TBT.nc --atom 20-30 --dos --ados Left --out dos_20-30.dat
sdata siesta.TBT.nc --atom 20-30[1-3] --dos --ados Left --out dos_20-30_1-3.dat
```

where the former is the total PDOS on atoms 20 through 30, and the latter is the PDOS on orbitals 1, 2 and 3 on atoms 20 through 30. It thus is extremely easy to extract different PDOS once the calculation has completed.

TBT.T.Bulk **false** *(logical)*

Calculate the bulk (pristine) electrode transmission if **true**.

This generates `SystemLabel.BTRANS_<>` and `SystemLabel.AVBTRANS_<>`.

NOTE: implicitly enables **TBT.DOS.Elecs** if **true**.

TBT.DOS.Elecs **false** *(logical)*

Calculate the bulk (pristine) electrode DOS if **true**.

This generates `SystemLabel.BDOS_<>` and `SystemLabel.AVBDOS_<>`.

NOTE: implicitly enables **TBT.T.Bulk** if **true**.

TBT.DOS.Gf **false** *(logical)*

depends on: **TBT.Atoms.Device**

Calculate the DOS from the Green function on the atoms in the device region:

$$\mathbf{G}(E) = [E\mathbf{S} - \mathbf{H} - \sum_{\mathfrak{e}} \mathbf{\Sigma}_{\mathfrak{e}}(E)]^{-1} \tag{4}$$

$$= \sum_{\mathfrak{e}} \mathbf{G}(E)\mathbf{\Gamma}_{\mathfrak{e}}(E)\mathbf{G}^{\dagger}(E) + \text{bound states} \tag{5}$$

$$= \sum_{\mathfrak{e}} \mathbf{A}_{\mathfrak{e}}(E) + \text{bound states} \tag{6}$$

NOTE: this flag should only be used if there are bound states in the scattering region (or if one wish to uncover whether there are bound states). Due to internal algorithms the DOS from the Green function is computationally more demanding than using **TBT.DOS.A** and **TBT.DOS.A.All**.

This generates `SystemLabel.DOS` and `SystemLabel.AVDOS`.

See **TBT.Atoms.Device.Connect**.

In case any of **TBT.DM.Gf**, **TBT.COOP.Gf** or **TBT.COHP.Gf** is **true** this flag will be set to **true** as well.

TBT.DOS.A **false** *(logical)*

depends on: **TBT.Atoms.Device**

Calculate the DOS from the spectral function. This will not calculate the DOS from the last electrode (last in the list **TBT.Elecs**), see **TBT.DOS.A.All**.

Its relation to the Green function DOS can be inferred from Eq. (4) (see **TBT.DOS.Gf**). If there are no bound states in the device region then prefer this option and **TBT.DOS.A.All**.

This generates `SystemLabel.ADOS_<>` and `SystemLabel.AVADOS_<>`.

See **TBT.Atoms.Device.Connect**.

In case any of **TBT.Current.Orb**, **TBT.DM.A**, **TBT.COOP.A** or **TBT.COHP.A** is **true** this flag will be set to **true** as well.

TBT.DOS.A.All **false** *(logical)*

depends on: **TBT.Atoms.Device**

Calculate the DOS from the spectral function and do so with *all* electrodes.

This additionally generates `SystemLabel.ADOS_<>` and `SystemLabel.AVADOS_<>` for the last electrode in **TBT.Elecs**.

NOTE: if **true**, this implicitly sets **TBT.DOS.A** to **true**.

Setting the flags **TBT.DOS.Gf** and **TBT.DOS.A.All** to **true** enables the estimation of bound states in the scattering region via this simple expression

$$\rho_{\text{bound-states}} = \rho_{\mathbf{G}} - \sum_{i} \rho_{\mathbf{A}_i}, \tag{7}$$

where the sum is over all electrodes, $\mathbf{G}$ and $\mathbf{A}_i$ are the Green and spectral function, respectively. Note that typically $\rho_{\text{bound-states}} = 0$.

The below two options enables the calculation of the energy resolved density matrices. In effect they may be used to construct LDOS(E) profiles using SISL.

TBT.DM.Gf **false** *(logical)*

depends on: **TBT.Atoms.Device**

Calculate the energy and k-resolved density matrix for the Green function. The density matrix may be used to construct real-space LDOS profiles.

TBT.DM.A **false** *(logical)*

depends on: **TBT.Atoms.Device**

Calculate the energy and k-resolved density matrix for the electrode spectral functions. The density matrix may be used to construct real-space LDOS profiles.

In addition to the DOS analysis of the Green and spectral functions, the Crystal Orbital Overlap Population and Crystal Orbital Hamilton Population may also be calculated. These are only available if TBtrans is compiled with NetCDF-4 support.

TBT.COOP.Gf **false** *(logical)*

depends on: **TBT.Atoms.Device**, **TBT.DOS.Gf**

Calculate COOP from the Green function in the device region.

The COOP curve is calculated as:

$$\mathrm{COOP}_{\mu\nu} = \frac{-1}{\pi}\Im[\mathbf{G}_{\mu\nu}\mathbf{S}_{\nu\mu}]. \tag{8}$$

The COOP curves are orbital, energy and k-resolved and they may thus result in very large output files.

NOTE: Untested!

TBT.COOP.A **false** *(logical)*

depends on: **TBT.Atoms.Device**, **TBT.DOS.A.All**

Calculate COOP from the spectral function in the device region.

The COOP curve is calculated as:

$$\mathrm{COOP}_{\mu\nu} = \frac{1}{2\pi}\Re[\mathbf{A}_{\mu\nu}\mathbf{S}_{\nu\mu}]. \tag{9}$$

The COOP curves are orbital, energy and k-resolved and they may thus result in very large output files.

NOTE: Untested!

TBT.COHP.Gf **false** *(logical)*

depends on: **TBT.Atoms.Device**

Calculate COHP from the Green function in the device region.

The COHP curve is calculated as:

$$\mathrm{COHP}_{\mu\nu} = \frac{-1}{\pi}\Im[\mathbf{G}_{\mu\nu}\mathbf{H}_{\nu\mu}]. \tag{10}$$

The COHP curves are orbital, energy and k-resolved and they may thus result in very large output files.

NOTE: Untested!

TBT.COHP.A **false** *(logical)*

depends on: **TBT.Atoms.Device**, **TBT.DOS.A.All**

Calculate COHP from the spectral function in the device region.

The COHP curve is calculated as:

$$\mathrm{COHP}_{\mu\nu} = \frac{1}{2\pi}\Re[\mathbf{A}_{\mu\nu}\mathbf{H}_{\nu\mu}]. \tag{11}$$

The COHP curves are orbital, energy and k-resolved and they may thus result in very large output files.

NOTE: Untested!

TBT.T.Eig 0 *(integer)*

Specify how many of the transmission eigenvalues will be calculated.

This generates `SystemLabel.TEIG_<1>_<2>` and `SystemLabel.AVTEIG_<1>_<2>`, possibly `SystemLabel.CEIG_<1>` and `SystemLabel.AVCEIG_<1>`. The former is for two different electrodes $i \neq j$, while the latter is for electrode $i = j$.

NOTE: if you specify a number of eigenvalues above the available number of eigenvalues, TBtrans will automatically truncate it to a reasonable number.

NOTE: The transmission eigenvalues for $N > 2$ systems is not fully understood and the transmission eigenvalues calculated in TBtrans is done by diagonalizing this sub-matrix:

$$\mathbf{G}\mathbf{\Gamma}_i\mathbf{G}^\dagger\mathbf{\Gamma}_j. \tag{12}$$

TBT.T.All false *(logical)*

By default TBtrans only calculates transmissions in *one* direction because time-reversal symmetry makes $T_{ij} = T_{ji}$. If one wishes to assert this, or if time-reversal symmetry does not apply for your system, one may set this to **true** to explicitly calculate all transmissions.

This additionally generates `SystemLabel.TRANS_<1>_<2>` and `SystemLabel.AVTRANS_<1>_<2>` for all electrode combinations (and the equivalent eigenvalue files if **TBT.T.Eig** is **true**.

TBT.T.Out false *(logical)*

The total transmission out of any electrode[1] may easily be calculated using only the scattering matrix of the origin electrode and the scattering region Green function. This enables the calculation of these equations

$$i\,\mathrm{Tr}[(\mathbf{G} - \mathbf{G}^\dagger)\mathbf{\Gamma}_j], \tag{13}$$

$$\mathrm{Tr}[\mathbf{G}\mathbf{\Gamma}_j\mathbf{G}^\dagger\mathbf{\Gamma}_j]. \tag{14}$$

The total transmission out of electrode j may then be calculated as

$$T_j = i\,\mathrm{Tr}[(\mathbf{G} - \mathbf{G}^\dagger)\mathbf{\Gamma}_j] - \mathrm{Tr}[\mathbf{G}\mathbf{\Gamma}_j\mathbf{G}^\dagger\mathbf{\Gamma}_j]. \tag{15}$$

This generates two sets of files: `SystemLabel.CORR_<>` and `SystemLabel.TRANS_<1>_<1>` which corresponds to equations Eqs. (13) and (14), respectively. To calculate T_j subtract the two files according to Eq. (15).

TBT.Current.Orb false *(logical)*

depends on: **TBT.Atoms.Device**, **TBT.DOS.A**

[1] In $N > 2$-electrode calculations one *cannot* use this quantity to calculate the total current out of an electrode.

Whether the orbital currents will be calculated and stored. These will be stored in a sparse matrix format corresponding to the SIESTA sparse format with only the device atoms in the sparse pattern.

Orbital currents are implemented as:

$$J_{\alpha\beta}(E) = i[(\mathbf{H}_{\beta\alpha} - E\mathbf{S}_{\beta\alpha})\mathbf{A}_{\alpha\beta}(E) - (\mathbf{H}_{\alpha\beta} - E\mathbf{S}_{\alpha\beta})\mathbf{A}_{\beta\alpha}(E)], \tag{16}$$

where we have left out the pre-factor $(e/\hbar)$ intentionally. SISL may be used to analyze the orbital currents and enables easy transformation of orbital currents to bond currents and activity currents[1].

NOTE: this requires TBTRANS to be compiled with NetCDF-4 support, see Sec. **??**.

TBT.Spin ⟨**all**⟩ *(integer)*

If the Hamiltonian is a polarized calculation one my define the index of the spin to be calculated.

This allows one to simultaneously calculate the spin-up and spin-down transmissions, for instance

```
$ tbtrans -fdf TBT.Spin:1 -D UP RUN.fdf &
$ tbtrans -fdf TBT.Spin:2 -D DOWN RUN.fdf &
```

which will create two folders `UP` and `DOWN` and output the relevant physical quantities in the respective folders.

TBT.Symmetry.TimeReversal **true** *(logical)*

Whether the Hamiltonian and the calculation should use time-reversal symmetry. Currently this only affects **k**-point sampling calculations by not removing any symmetry **k**-points.

If one has **k**-point sampling and wishes to use **TBT.Current.Orb** this should be **false**.

4.2.1 Device region

The scattering region (and thus device region) is formally consisting of all atoms besides the electrodes. However, when calculating the transmission this choice is very inefficient. Thus to heavily increase throughput one may define a smaller device region consisting of a subset of atoms in the scattering region.

The choice of atoms *must* separate each electrode from each other. TBTRANS will stop if this is not enforced.

Remark that the physical quantities such as DOS, spectral DOS, orbital currents may only be calculated in the selected device region.

TBT.Atoms.Device ⟨**all but electrodes**⟩ *(block/list)*

This flag may either be a block, or a list.

A block with each line denoting the atoms that consists of the device region.

```
%block TBT.Atoms.Device
   atom [ 10 -- 20 ]
   atom [ 30 -- 40 ]
   # Atoms removed from the device region
   # Even though they are specified in other
   # lines
```

```
   not-atom [ 15, 35]
% endblock
# Or equivalently as a list
TBT.Atoms.Device [10 -- 14, 16 -- 20, 30 -- 34, 36 -- 40]
```

will limit the device region to atoms [10–14, 16–20, 30–34, 36–40].

TBT.Atoms.Device.Connect **false** *(logical)*

Setting this to **true** will *extend* the device region to also include atoms that the input device atoms has matrix elements between. This may be important when using non-orthogonal basis sets as one can ensure the full overlap matrix on the selected device atoms.

NOTE: this parameter should be set to **true** in case accurate DOS calculations are required on the specified device atoms (if using a non-orthogonal basis set).

TBT.Atoms.Buffer ⟨**None**⟩ *(block/list)*

A block with each line denoting the atoms that are disregarded in the Green function calculation. For self-consistent calculations it may be required to introduce buffer atoms which are removed from the SCF cycle. In such cases these atoms should also be removed from the transport calculation.

```
%block TBT.Atoms.Buffer
   atom [ 1 -- 5 ]
%endblock
# Or equivalently as a list
TBT.Atoms.Buffer [1 -- 5]
```

will remove atoms [1–5] from the calculation.

4.2.2 Brillouin zone

TBtrans allows calculating physical quantities via averaging in the Brillouin zone.

TBT.k ⟨**kgrid_Monkhorst_Pack**⟩ *(list/block)*

depends on: **TBT.k.File**

If **TBT.k.File** is present, that will have precedence.

Specify how to perform Brillouin zone integrations.

This may be given as a list like this:

```
TBT.k [A B C]
```

where each integer corresponds to the diagonal elements of the Monkhorst-Pack grid. I.e.

```
TBT.k [10 10 1]
%block TBT.k
  10  0 0 0.
   0 10 0 0.
   0  0 1 0.
%endblock
```

are equivalent.

If you supply this flag as a block the following options are available:

path Define a Brillouin zone path[2] where the k-points are equi-spaced. It may be best described using this example:

```
path 10
  from 0.  0.  0.
  to   0.5 0.5 0.
path 20
  from 0.25 0.25 0.
  to   0.0  0.5  0.
```

This will create k-points starting from the Γ-point and move to the Brillouin zone boundary at [1/2, 1/2, 0] with spacing to have 10 points.

There is no requirement that the **path**s are connected and one may specify as many paths as wanted.

even-path It is generally advised to add this flag in the blog (somewhere) if one wants equi-distance k-spacings in the Brillouin zone. This flag sums up the total number of k-points on the total path and then calculates the exact number of required points required on each path to have the same δk in each path.

NOTE: if any one **path** is found in the block the options (explained below) are ignored.

diagonal|diag Specify the number of k points in each unit-cell direction

diagonal 3 3 1 will use 3 k points along the first and second lattice vectors and only one along the third lattice vector.

displacement|displ Specify the displacement of the Brillouin zone k points along each lattice vector. This input is similar to **diagonal** but requires real input.

displacement 0.5 0.25 0. will displace the first and second k origin to [1/2, 1/4, 0].

size This reduces the sampled Brillouin zone to only the fractional size of each lattice vector direction.

This may be used to only sample k-points in a reduced Brillouin zone which for instance is useful if one wishes to sample the Dirac point in graphene in an energy range of −0.5 eV – 0.5 eV.

size 0.5 1. 1. will reduce the sampled k points along the first reciprocal lattice to be in the range]−1/4, 1/4], while the other directions are still sampled]−1/2, 1/2].

NOTE: expert use only.

list Explicitly specify the sampled k-points and (optionally) the associated weights.

```
list 2
  0.  0.  0.  0.5
  0.5 0.5 0.
```

where the integer on the **list** line specifies the number of lines that contains k points. Each line *must* be created with 3 reals which define the k point in units of the reciprocal lattice vectors (]−1/2–1/2]).

An optional 4th value denote the associated weight which is defaulted to $1/N$ where N is the total number of k points.

NOTE: if this is found it will neglect the other input options (except **path**).

[2] Much like **BandLines** in SIESTA.

method Define how the k-points should be created in the Brillouin zone.

Currently these options are available (**Monkhorst-Pack** being the default)

Monkhorst-Pack|MP Use the regular Monkhorst-Pack sampling (equi-spaced) with simple linear weights.

Gauss-Legendre Use the Gauss-Legendre quadrature and weights for constructing the k points and weights. These k points are not equi-spaced and puts more weight to the Γ point.

Simpson-mix Use the Newton-Cotes method (Simpson, degree 3) which uses equi-spaced points but non-uniform weights.

Boole-mix Use the Newton-Cotes method (Boole, degree 5) which uses equi-spaced points but non-uniform weights.

⟨**siesta-method**⟩ One may also use the typical **kgrid_Monkhorst_Pack** method of input as done in SIESTA. This is a 3×3 block such as:

```
10  0  0 0.
 0 15  0 0.
 0  0  1 0.
```

which uses 10, 15 and 1 k-points along the 1st, 2nd and 3rd reciprocal lattice vectors. And with 0 displacement.

NOTE: it is recommended to use the **diagonal** option unless off-diagonal k points are needed.

TBT.k.File **NONE** *(string)*

If provided, it is a (relative) path to a file containing k-points in the filename containing points in units of the reciprocal lattice vectors. I.e. in the range of $]-1/2; 1/2]$. See e.g. **TBT.k.list** for details, or see **kgrid.File** in the SIESTA manual for more details about the exact file layout.

The only exception is that the weights need not sum to 1 as is the case for SIESTA. This is because TBtrans can do reduced Brillouin zone calculations. Hence, be careful about what you pass.

4.2.3 Energy grid

TBtrans uses a default energy reference as the Fermi level in the corresponding TranSIESTA calculation. I.e. the equilibrium Fermi level. Thus one should be aware when using a shifted bias window that the calculated properties shifts according to the applied bias. For example; if one performs two equivalent 2-terminal calculations A) with $\mu_L = V$, $\mu_R = 0$ and the other B) with $\mu_L = V/2$, $\mu_R = -V/2$ then the calculated properties are equivalent if one shifts the energy spectrum of A) by $E \to E - V/2$. Any 2-terminal calculation is recommended to be setup with $\mu_L = V/2$ and $\mu_R = -V/2$ due to the fixed energy reference, $E_R = 0$.

The Green function is calculated at explicit energies and does not rely on diagonalization routines to retrieve the eigenspectrum. This is due to the smearing of states from the coupling with the semi-infinite electrodes.

It is thus important to define an energy grid for analysis of the DOS and transmission.

TBT.Contours.Eta $\min[\eta_e]/10$ *(energy)*

depends on: **TBT.Elecs.Eta**

The imaginary (η) part of the Green function in the device region. Note that the electrodes imaginary part may be controlled via **TBT.Elecs.Eta**.

This value controls the smearing of the DOS on the energy axis. Generally one need not take into account η values different from 0. However, in cases where localized states are found a smearing in the device region can help numerics. Therefore it defaults to $\min[\eta_{\mathbf{e}}]/10$. This ensures that the device broadening is always smaller than the electrodes while allowing broadening of localized states.

%block TBT.Contours ***see note further down*** *(block)*

Each line in this block corresponds to a specific contour. Enabling several lines of input allows to create regions of the energy grid which has a high density and ranges of energies with lower density. Also it allows to bypass energy ranges where the DOS is zero in for instance a semiconductor.

See **TBT.Contour.<>** for details on specifying the energy contour.

%block TBT.Contour.<> ⟨**None**⟩ *(block)*

Specify a contour named <> with options within the block.

The names <> are taken from the **TBT.Contours** block.

The format of this block is made up of at least 3 lines, in the following order of appearance.

from *a* to *b* Define the integration range on the energy axis. Thus a and b are energies.

points|delta|file Define the number of integration points/energy separation. If specifying the number of points an integer should be supplied.

If specifying the separation between consecutive points an energy should be supplied (e.g. **0.01 eV**).

Optionally one may specify a file which contains the energy points and their weights.

This file has the same formatting as the `SystemLabel.TBT.CC` output with some optional inputs. Below is an example input file.

```
# There are 2 different input options:
# 1. Re[E] Im[E] W  (optional unit)
# 2. Re[E] W        (optional unit) (imaginary part will be device Eta)
# If the unit is specified on any line, all subsequent lines will use
# the specified unit. Default unit is eV!
# Empty lines and lines starting with # will be ignored.
-0.5  0.1    # E = -0.5 eV, weight (for integrating current) of 0.1 eV
-0.01 0.1 Ry # E = -0.01 Ry and weight 0.1 Ry
-0.02 0.1    # E = -0.02 Ry (above unit continue) and weight 0.1 Ry
-0.2  0.1 eV # E = -0.2 eV and weight 0.1 eV
-0.2 1. 0.1  # E = -0.2 eV and 1. eV eta and weight 0.1 eV
```

If the file specified is `SystemLabel.TBT.CC` the same energy points will be used. Note that the resulting `SystemLabel.TBT.nc` file does not store the energies as complex numbers, thus one cannot subsequently extract the η value used for the individual energy points.

NOTE: for PHtrans the energies will be squared internally to be in correct units, hence the units should still be eV.

method Specify the numerical method used to conduct the integration. Here a number of different numerical integration schemes are accessible

mid|mid-rule Use the mid-rule for integration.

simpson|simpson-mix Use the composite Simpson 3/8 rule (three point Newton-Cotes).

boole|boole-mix Use the composite Booles rule (five point Newton-Cotes).

G-legendre Gauss-Legendre quadrature.

tanh-sinh Tanh-Sinh quadrature.

NOTE: has **opt precision <>**.

user User defined input via a file.

opt Specify additional options for the **method**. Only a selected subset of the methods have additional options.

By default the TBtrans energy grid is defined as

```
TBT.Contours.Eta 0. eV
%block TBT.Contours
  line
%endblock
%block TBT.Contour.line
   from -2. eV to 2. eV
    delta 0.01 eV
      method mid-rule
%endblock
```

An example of input using a file (note that regular contour setups may be used together with file-inputs)

```
TBT.Contours.Eta 0. eV
%block TBT.Contours
  file
%endblock
%block TBT.Contour.file
   from 2. eV to 2.5 eV
    file my_energies
%endblock
```

Note that the energy specifications are necessary (due to internal bookkeeping).

4.3 Chemical potentials

For N electrodes there will also be N_μ chemical potentials. They are defined via blocks similar to **TBT.Elecs**. If no bias is applied TBtrans will default to a single chemical potential with the chemical potential in equilibrium. In this case you need not specify any chemical potentials.

By default TBtrans creates a single chemical potential with the chemical potential equal to the device Fermi-level. Hence, performing non-bias calculations does not require one to specify these blocks.

%block TBT.ChemPots ⟨**None**⟩ *(block)*

Each line denotes a new chemical potential which may is further defined in the **TBT.ChemPot.<>** block.

%block TBT.ChemPot.<> ⟨**None**⟩ *(block)*

Each line defines a setting for the chemical potential named <>.

chemical-shift|mu Define the chemical shift (an energy) for this chemical potential. One may specify the shift in terms of the applied bias using **V/<integer>** instead of explicitly typing the energy.

ElectronicTemperature|Temp|kT Specify the electronic temperature (as an energy or in Kelvin). This defaults to **TS.ElectronicTemperature**.

One may specify this in units of **TS.ElectronicTemperature** by using the unit **kT**.

It is important to realize that the parameterization of the voltage into the chemical potentials enables one to have a *single* input file which is never required to be changed, even when changing the applied bias.

These options complicate the input sequence for regular 2 electrode which is unfortunate.

4.4 Electrode configuration

The electrodes are defining the semi-infinite region that is coupled to the scattering region.

TBtrans is a fully N electrode calculator. Thus the input for such setups is rather complicated.

TBtrans defaults to read the TranSIESTA electrodes and as such one may replace **TBT** by **TS** and TBtrans will still work. However, the **TBT** has precedence.

If there is only 1 chemical potential all electrodes will default to use this chemical potential, thus for non-bias calculations there is no need to specify the chemical potential (**TBT.Elec.<>.chemical-potential**).

%block TBT.Elecs ⟨**None**⟩ *(block)*

Each line denote an electrode which may be queried in **TBT.Elec.<>** for its setup.

%block TBT.Elec.<> ⟨**None**⟩ *(block)*

Each line represents a setting for electrode <>. There are a few lines that *must* be present, **HS**, **semi-inf-dir**, **electrode-pos**, **chem-pot** (only if **TBT.Voltage** is not 0).

If there are some settings that you only want to take effect in TBtrans calculations you can prefix the option with **tbt.Eta**, e.g. which will only be used for the TBtrans calculations. Note that *all* **tbt.*** options must be located at the end of the block. This may be particularly useful with respect to the Green function file options.

HS The electronic structure information from the initial electrode calculation. This file retains the geometrical information as well as the Hamiltonian, overlap matrix and the Fermi-level of the electrode. This is a file-path and the electrode `SystemLabel.TSHS` need not be located in the simulation folder.

TBtrans also reads NetCDF4 files which contain the electronic structure. This may be created using sisl.

NOTE: Please note that TranSIESTA expects a metallic electrode. Results can not be trusted for semi-conductors.

semi-inf-direction|semi-inf-dir|semi-inf The semi-infinite direction of the electrode with re-

spect to the electrode unit-cell.

It may be one of **[-+][abc]**, **[-+]A[123]**, **ab**, **ac**, **bc** or **abc**. The latter four all describe a real-space self-energy as described in [2].

NOTE: this has nothing to do with the scattering region unit cell, TBtrans will figure out the alignment of the electrode unit-cell and the scattering region unit-cell.

chemical-potential|chem-pot|mu The chemical potential that is associated with this electrode. This is a string that should be present in the **TBT.ChemPots** block in case there is a bias applied in the calculation.

electrode-position|elec-pos The index of the electrode in the scattering region. This may be given by either **elec-pos <idx>**, which refers to the first atomic index of the electrode residing at index **<idx>**. Else the electrode position may be given via **elec-pos end <idx>** where the last index of the electrode will be located at **<idx>**.

used-atoms *depends on:* **TBT.Elec.<>.semi-inf-direction**

Number of atoms from the electrode calculation that is used in the scattering region as electrode. This may be useful when the periodicity of the electrodes forces extensive electrodes in the semi-infinite direction.

If the semi-infinite direction is *positive*, the first atoms will be retained. Contrary, if the semi-infinite direction is *negative*, the last atoms will be retained.

NOTE: do not set this if you use all atoms in the electrode.

Bulk Logical controlling whether the Hamiltonian of the electrode region in the scattering region is enforced *bulk* or whether the Hamiltonian is taken from the scattering region elements.

Eta *depends on:* **TBT.Elecs.Eta**

Control the imaginary energy (η) of the surface Green function for this electrode.

NOTE: if this energy is negative the complex value associated with the contour is used. This is particularly useful when providing a user-defined contour. Ensure that all imaginary values are larger than 0 as otherwise TBtrans may seg-fault.

NOTE: for PHtrans calculations you are highly encouraged to change this value since the default (1 meV) is very low.

Bloch 3 integers are present on this line which each denote the number of times bigger the scattering region electrode is compared to the electrode, in each lattice direction. Remark that these expansion coefficients are with regard to the electrode unit-cell. This is denoted "Bloch" because it is an expansion based on Bloch waves.

Please see *Matching electrode coordinates: basic rules* in the SIESTA manual for details.

Bloch-A/a1|B/a2|C/a3 Specific Bloch expansions in each of the electrode unit-cell direction. See **Bloch** for details.

out-of-core *depends on:* **TBT.Elec.<>.Gf**

If **true** the GF files are created which contain the surface Green function. Setting this to **true** may be advantageous when performing many calculations using the same k and energy grid using the same electrode. In those case this will heavily increase throughput. If **false** (default) the surface Green function will be calculated when needed.

NOTE: simultaneous calculations may read the same GF file.

tbt.Gf/Gf *depends on:* **TBT.Elec.<>.Out-of-core**

String with filename of the surface Green function data. This may be used to place a common surface Green function file in a top directory which may then be used in all calculations using the same electrode and the same contour. If doing many calculations with the same electrode and $\mathbf{k}$, E grids, then this can greatly improve throughput. It has a cost of disk-space. Note that the energy-grids are dependent on the applied bias.

Gf-Reuse *depends on:* **TBT.Elec.<>.out-of-core**

Logical deciding whether the surface Green function file should be re-used or deleted. If this is **false** the surface Green function file is deleted and re-created upon start.

pre-expand *depends on:* **TBT.Elec.<>.out-of-core**

String denoting how the expansion of the surface Green function file will be performed. This only affects the Green function file if **Bloch** is larger than 1. By default the Green function file will contain the fully expanded surface Green function, but not Hamiltonian and overlap matrices (**Green**). One may reduce the file size by setting this to **Green** which only expands the surface Green function. Finally **none** may be passed to reduce the file size to the bare minimum. For performance reasons **all** is preferred.

Accuracy *depends on:* **TBT.Elecs.Accuracy**

Control the convergence accuracy required for the self-energy calculation when using the Lopez-Sanchez, Lopez-Sanchez iterative scheme.

NOTE: advanced use *only*.

delta-Ef Specify an offset for the Fermi-level of the electrode. This will directly be added to the Fermi-level found in the electrode file.

NOTE: this option only makes sense for semi-conducting electrodes since it shifts the entire electronic structure. This is because the Fermi-level may be arbitrarily placed anywhere in the band gap. It is the users responsibility to define a value which does not introduce a potential drop between the electrode and device region.

V-fraction Specify the fraction of the chemical potential shift in the electrode-device coupling region. This corresponds to:

$$\mathbf{H}_{\mathfrak{e}D} \leftarrow \mathbf{H}_{\mathfrak{e}D} + \mu_{\mathfrak{e}} \mathrm{V}_f \mathbf{S}_{\mathfrak{e}D} \tag{17}$$

in the coupling region. Consequently the value *must* be between 0 and 1.

NOTE: this option may be used for tight-binding calculations as an empirical applied bias (with the potential drop at the electrode/device interface). It *should not* be used for converged TranSIESTA calculations.

check-kgrid For N electrode calculations the $\mathbf{k}$ mesh will sometimes not be equivalent for the electrodes and the device region calculations. However, TBtrans requires that the device and electrode $\mathbf{k}$ samplings are commensurate. This flag controls whether this check is enforced.

NOTE: only use if fully aware of the implications (for tight-binding calculations this may safely be set to **false**).

There are several flags which are globally controlling the variables for the electrodes (with **TBT.Elec.<>** taking precedence).

TBT.Elecs.Bulk **true** *(logical)*

This globally controls how the Hamiltonian is treated in all electrodes. See **TBT.Elec.<>.Bulk**.

TBT.Elecs.Eta 1 meV *(energy)*

Globally control the imaginary energy (η) used for the surface Green function calculation. This η value is *not* used in the device region. See **TBT.Elec.<>.Eta** for extended details on the usage of this flag.

TBT.Elecs.Accuracy 10^{-13} eV *(energy)*

Globally control the accuracy required for convergence of the self-energy. See **TBT.Elec.<>.Accuracy**.

TBT.Elecs.Neglect.Principal **false** *(logical)*

If this is **false** TBtrans dies if there are connections beyond the principal cell.

NOTE: set this to **true** with care, non-physical results may arise. Use at your own risk!

TBT.Elecs.Out-of-core **false** *(logical)*

This enables reusing the self-energies by storing them on-disk (**true**). The surface Green function files may be large files but heavily increases throughput if one performs several transport calculations using the same electrodes.

You are encouraged to set this to **true** to reduce computations. See **TBT.Elec.<>.out-of-core**.

Currently this option is not compatible with **TBT.T.Bulk** and **TBT.DOS.Elecs**, and the bulk transmission and bulk DOS will not be calculated if this option is **true**.

TBT.Elecs.Gf.Reuse **true** *(logical)*

depends on: **TBT.Elecs.Out-of-core**

Globally control whether the surface Green function files should be re-used (**true**) or re-created (**false**). See **TBT.Elec.<>.Gf-Reuse**.

TBT.Elecs.Coord.EPS 10^{-4} Bohr *(length)*

When using Bloch expansion of the self-energies one may experience difficulties in obtaining perfectly aligned electrode coordinates.

This parameter controls how strict the criteria for equivalent atomic coordinates is. If TBtrans crashes due to mismatch between the electrode atomic coordinates and the scattering region calculation, one may increase this criteria. This should only be done if one is sure that the atomic coordinates are almost similar and that the difference in electronic structures of the two may be negligible.

4.4.1 Principal layer interactions

It is *extremely* important that the electrodes only interact with one neighboring supercell due to the self-energy calculation. TBtrans will print out a block as this

```
<> principal cell is perfect!
```

if the electrode is correctly setup and it only interacts with its neighboring supercell. In case the electrode is erroneously setup, something similar to the following will be shown in the output file.

```
<> principal cell is extending out with 96 elements:
   Atom 1 connects with atom 3
```

```
Orbital 8 connects with orbital 26
Hamiltonian value: |H(8,6587)|@R=-2 =  0.651E-13 eV
Overlap          :  S(8,6587)|@R=-2 =   0.00
```

It is imperative that you have a *perfect* electrode as otherwise nonphysical results will occur.

4.5 Calculation settings

The calculation time is currently governed by two things:

1. the size of the device region,
2. and by the partitioning of the block-tri-diagonal matrix.

The first may be controlled via **TBT.Atoms.Device**. If one is only interested in transmission coefficients this flag is encouraged to select the minimum number of atoms that will successfully run the calculation. Please see the flag entry for further details.

Secondly there is, currently, no way to determine the most optimal block-partitioning of a banded matrix and TBtrans allows several algorithms to determine an optimal partitioning scheme. The following flag controls the partitioning for the device region.

TBT.Analyze **false** *(logical)*

As the pivoting algorithm *highly* influences the performance and throughput of the transport calculation it is crucial to select the best performing algorithm available. This option tells TBtrans to analyze the pivoting table for nearly all the implemented algorithms and print-out information about them.

NOTE: we advice users to *always* run an analyzation step prior to actual calculation and select the *best* BTD format. This analyzing step is very fast and can be performed on small work-station computers, even on systems of $\gg 10,000$ orbitals.

To run the analyzing step you may do:

```
tbtrans -fdf TBT.Analyze RUN.fdf > analyze.out
```

note that there is little gain on using MPI and it should complete within a few minutes, no matter the number of orbitals.

Choosing the best one may be difficult. Generally one should choose the pivoting scheme that uses the least amount of memory. However, one should also choose the method with the largest block-size being as small as possible. As an example:

```
TBT.BTD.Pivot.Device atom+GPS
...
    BTD partitions (7):
     [ 2984, 2776, 192, 192, 1639, 4050, 105 ]
    BTD matrix block size [max] / [average]: 4050 /   1705.429
    BTD matrix elements in % of full matrix:   47.88707 %

TBT.BTD.Pivot.Device atom+GGPS
...
    BTD partitions (6):
     [ 2880, 2916, 174, 174, 2884, 2910 ]
    BTD matrix block size [max] / [average]: 2916 /   1989.667
```

```
BTD matrix elements in % of full matrix:   48.62867 %
```

Although the GPS method uses the least amount of memory, the GGPS will likely perform better as the largest block in GPS is 4050 vs. 2916 for the GGPS method.

TBT.BTD.Optimize **speed|memory** *(string)*

When selecting the smallest blocks for the BTD matrix there are certain criteria that may change the size of each block. For very memory consuming jobs one may choose the **memory**.

NOTE: often both methods provide *exactly* the same BTD matrix due to constraints on the matrix.

TBT.BTD.Pivot.Device **atom-⟨largest overlapping electrode⟩** *(string)*

Decide on the partitioning for the BTD matrix. One may denote either **atom+** or **orb+** as a prefix which does the analysis on the atomic sparsity pattern or the full orbital sparsity pattern, respectively. If neither are used it will default to **atom+**.

<elec-name>|CG-<elec-name> The partitioning will be a connectivity graph starting from the electrode denoted by the name. This name *must* be found in the **TBT.Elecs** block. One can append more than one electrode to simultaneously start from more than 1 electrode. This may be necessary for multi-terminal calculations.

NOTE: One may append an optional setting **front** or **fan** which makes the connectivity graph to consider the geometric front of the atoms. For extreme scale simulations or tight-binding calculations with constrictions this may improve the BTD matrix substantially because it splits the unit-cell into segments of equal width.

rev-CM Use the reverse Cuthill-McKee for pivoting the matrix elements to reduce bandwidth. One may omit **rev-** to use the standard Cuthill-McKee algorithm (not recommended).

This pivoting scheme depends on the initial starting electrodes, append **+<elec-name>** to start the Cuthill-McKee algorithm from the specified electrode.

GPS Use the Gibbs-Poole-Stockmeyer algorithm for reducing the bandwidth.

GGPS Use the generalized Gibbs-Poole-Stockmeyer algorithm for reducing the bandwidth.

PCG Use the perphiral connectivity graph algorithm for reducing the bandwidth.

This pivoting scheme *may* depend on the initial starting electrode(s), append **+<elec-name>** to initialize the PCG algorithm from the specified electrode.

Examples are

```
TBT.BTD.Pivot.Device atom+GGPS
TBT.BTD.Pivot.Device GGPS
TBT.BTD.Pivot.Device orb+GGPS
TBT.BTD.Pivot.Device orb+PCG
TBT.BTD.Pivot.Device orb+PCG+Left
TBT.BTD.Pivot.Device orb+rev-CM+Right
```

where the first two are equivalent. The 3rd and 4th are more heavily on analysis and will typically not improve the bandwidth reduction.

TBT.BTD.Pivot.Elec.<> **atom-⟨<>⟩** *(string)*

depends on: **TBT.Atoms.Device**

If **TBT.Atoms.Device** has been set to a reduced region the electrode self-energies must be *down-folded* through the atoms not part of the device-region. In this case these down-fold regions can also be considered a BTD matrix which may be optimized separately from the device region BTD matrix.

This option have all available options as described in **TBT.BTD.Pivot.Device** but one will generally find the best pivoting scheme by using the default (**atom-<>**) which is the atomic connectivity graph from the electrode it-self.

It may be advantageous to use **atom-<>-front** for very large tight-binding calculations where the device region is chosen far from this electrode and normal to the electrode-plane.

TBT.BTD.Spectral **propagation|column** *(string)*

Method used for calculating the spectral function ($\mathbf{A}_i$). For 4 or more electrodes the **column** option is the default while **propagation** is the default for less electrodes.

NOTE: this option may heavily influence performance. Test for a single k-point and single energy point to figure out the implications of using one over the other.

TBT.BTD.Pivot.Graphviz **false** *(logical)*

Create Graphviz[3] compatible input files for the pivoting tables for all electrodes, `SystemLabel.TBT.<elec>.gv`, and the device, `SystemLabel.TBT.gv`.

These files may be processed by Graphviz display commands `neato` etc.

```
neato -x <>
neato -x -Tpdf <> -o graph.pdf
neato -x -Tpng <> -o graph.png
```

4.6 Input/Output

TBtrans IO is mainly relying on the NetCDF4 library and full capability is only achieved if compiled with this library.

Several fdf-flags control how TBtrans performs IO.

TBT.CDF.Precision **single|float|double** *(string)*

Specify the precision used for storing the quantities in the NetCDF4.

single takes half the disk-space as **double** and will generally retain a sufficient precision of the quantities.

single and **float** are equivalent.

NOTE: all calculations are performed using **double** so this is *only* a storage precision.

TBT.CDF.DOS.Precision **<TBT.CDF.Precision>** *(string)*

Specify the precision used for storing DOS in NetCDF4.

TBT.CDF.T.Precision **<TBT.CDF.Precision>** *(string)*

Specify the precision used for storing transmission function in NetCDF4.

TBT.CDF.T.Eig.Precision **<TBT.CDF.Precision>** *(string)*

Specify the precision used for storing transmission eigenvalues in NetCDF4.

[3] `www.graphviz.org`

TBT.CDF.Current.Precision <TBT.CDF.Precision> *(string)*

Specify the precision used for storing orbital current in NetCDF4.

NOTE: This is heavily advised to be in single precision as this may easily use large amounts of disk-space if in double precision.

TBT.CDF.DM.Precision <TBT.CDF.Precision> *(string)*

Specify the precision used for storing density matrices in NetCDF4.

NOTE: This is heavily advised to be in single precision as this may easily use large amounts of disk-space if in double precision.

TBT.CDF.COOP.Precision <TBT.CDF.Precision> *(string)*

Specify the precision used for storing COOP and COHP curves in NetCDF4.

NOTE: This is heavily advised to be in single precision as this may easily use large amounts of disk-space if in double precision.

TBT.CDF.Compress 0 *(integer)*

Specify whether the NetCDF4 files stored will be compressed. This may heavily reduce disk-utilization at the cost of some performance.

This number must be between 0 (no compression) and 9 (maximum compression). A higher compression is more time consuming and a good compromise between speed and compression is 3.

NOTE: one may subsequently to a TBtrans compilation compress a NetCDF4 file using:

```
nccopy -d 3 siesta.TBT.nc newsiesta.TBT.nc
```

NOTE: one *can not* do parallel I/O together with compression.

TBT.CDF.MPI false *(logical)*

Whether the IO is performed in parallel. If using a large amount of MPI processors this may increase performance.

NOTE: the actual performance increase is *very* dependent on your hardware support for parallel IO.

NOTE: this automatically sets the compression to 0 (one cannot compress and perform parallel IO).

4.6.1 Self-energy

TBtrans enables the storage of the self-energies from the electrodes in selected regions. I.e. in a two electrode setup the self-energies may be "down-folded" to a region of interest (say molecule etc.) and then saved.

This feature enables one to easily use self-energies in Python for subsequent analysis etc. It is only available if compiled against NetCDF4.

TBT.SelfEnergy.Save false *(logical)*

Store the self-energies of the electrodes. The self-energies are first down-folded into the device region (see **TBT.Atoms.Device**).

TBT.SelfEnergy.Save.Mean false *(logical)*

If **true** the down-folded self-energies will be k-averaged after TBtrans has finished.

TBT.SelfEnergy.Only **false** *(logical)*

If **true** this will *only* calculate and store the down-folded self-energies. No physical quantities will be calculated and TBtrans will quit.

TBT.CDF.SelfEnergy.Precision **<TBT.CDF.Precision>** *(string)*

depends on: **TBT.CDF.Precision**

Specify the precision used for storing the self-energies in NetCDF4.

TBT.CDF.SelfEnergy.Compress **<TBT.CDF.Compress>** *(integer)*

depends on: **TBT.CDF.Compress**

Specify the compression of the self-energies in NetCDF4.

TBT.CDF.SelfEnergy.MPI **false** *(logical)*

depends on: **TBT.CDF.MPI**

If **true** TBtrans will use MPI when writing the NetCDF files containing the downfolded self-energies.

4.6.2 Projected transmissions

The transmission through a scattering region is determined by the electrodes band-structure and the energy levels for the scattering part. In for instance molecular electronics it is often useful to determine which molecular orbitals are responsible for the transmission as well as knowing their hybridization with the substrate (electrodes).

TBtrans implements an advanced projection method which splits the transmission and DOS into eigenstate projectors.

In the following we concentrate on a 2 terminal device while it may be used for N electrode calculations. One important aspect of projection is that the self-energies that are to be projected *must* be fully located on the projection region. TBtrans will die if this is not enforced. A projection can *only* be performed if the down-folding of the self-energies for the projected electrode is fully encapsulated in the device region (**TBT.Atoms.Device**). I.e. one should reduce the device region such that any couplings from the electrodes only couple into the projection region. Generally for the most simple projections the device region should be equivalent to the projection region in case there is only one projection region.

These projections should not be confused with local DOS which may be obtained if compiled with the NetCDF4 library and via the use of sisl, see Sec. 4.2.

NOTE: if the **TBT.Projs** block is defined, then the **TBT.Projs.T** block is required in the input unless **TBT.Projs.Init** is **true**.

%block TBT.Projs ⟨**None**⟩ *(block)*

List of molecular projections used:

```
%block TBT.Projs
  M-L
  M-R
%endblock
```

This tells TBtrans that two projections will exist. Each projection setup will be read in **TBT.Proj.<>**.

There is no limit to the number of projection molecules.

%block TBT.Proj.<> ⟨**None**⟩ *(block)*

Block that designates a molecular projection by the names specified in the **TBT.Projs** block.

This block determines how each projection is interpreted, it consists of several options defined below:

atom There may be several **atom** lines. The full set of atomic indices will be used as a sub-space for the Hamiltonian. The atoms may be defined via these variants

atom *A* [***B*** [***C*** [**...**]]] A sequence of atomic indices which are used for the projection.

atom from *A* to *B* [**step *s***] Here atoms A up to and including B are used. If **step <s>** is given, the range A:B will be taken in steps of s.

```
atom from 3 to 10 step 2
```

will add atoms 3, 5, 7 and 9.

atom from *A* plus/minus *B* [**step *s***] Atoms A up to and including $A + B - 1$ are added to the projection. If **step <s>** is given, the range A:$A + B - 1$ will be taken in steps of s.

atom [**<A>, *B* -- *C*** [**step *s***], ***D***] Equivalent to **from ... to** specification, however in a shorter variant. Note that the list may contain arbitrary number of ranges and/or individual indices.

```
atom [2, 3 -- 10 step 2, 6]
```

will add atoms 2, 3, 5, 7, 9 and 6.

Gamma Logical variable which determines whether the projectors are the Γ-point projectors, or the k resolved ones. For Γ-only calculations this has no effect. If the eigenstates are non-dispersive in the Brillouin zone there should be no difference between **true** or **false**.

NOTE: it is *very* important to know the dispersion and possible band-crossings of the eigenstates if this option is **false**. For band-crossings one must manually perform the projections for the k-points in a stringent manner as the order of eigenstates are not retained.

proj <P-name> Allows to define a projection based on the eigenstates for the current molecule.

The **<P-name>** designates the name associated with this projection.

It is parsed like this, in the following 0 is the Fermi level (HOMO = −1, LUMO = 1):

level from <E1> to <E2> Energy eigenstates **E1** and **E2** will be part of the molecular orbitals that constitute this projection

level from <E> plus <N> Energy eigenstates between **E** and **E** + N − 1 will be part of the molecular orbitals that constitute this projection

level from <E> minus <N> Energy eigenstates between **E** and **E** − N + 1 will be part of the molecular orbitals that constitute this projection

level <E1> <E2> ... <En> All eigenstates specified will be part of the molecular orbitals that constitute this projection

level [**<list>**] A comma-separated list specification.

end All gathered eigenstates so far will constitute the projection named **<P-name>**

Note that level 0 refers to the Fermi level, it will be silently removed as it is not an eigenstate, so you do not need to think about it.

You can specify named projection blocks as many times as you want.

To conclude the full projection block here is an example describing three different projections for the left molecule in

```
%block TBT.Proj.M-L
  # We have 2 atoms on this molecule
  atom from 5 plus 2
  # We only do a Gamma projection
  Gamma .true.
  # We will utilise three different projections on
  # this molecule
  proj HOMO
   level -1
  end
  proj LUMO
   level 1
  end
  proj H-plus-L
   level from -1 to 1
  end
%endblock
```

Similarly for the right molecule we do

```
%block TBT.Proj.M-R
  # We have 2 atoms on this molecule
  atom from 8 plus 2
  # We only do a Gamma projection
  Gamma .true.
  # We will utilise three different projections on
  # this molecule
  proj HOMO
   level -1
  end
  proj LUMO
   level 1
  end
  proj H-plus-L
   level from -1 to 1
  end
%endblock
```

TBT.Proj.<>.States **false** *(logical)*

Save all states for the projection. The saved quantity can be post-processed to decipher the locality of each projection.

In the NetCDF file there will be two variables: `state` and `states` where the former will contain $\mathbf{S}^{1/2}|i\rangle$, while the latter will contain $|i\rangle$.

Needed if you wish to select specific molecular orbitals dependent on the nature of the molecular orbital.

TBT.CDF.Proj.Compress <TBT.CDF.Compress> *(integer)*

Allows a different compression for the projection file. The projection file is typically larger than the default output file, so compression of them separately might be needed.

TBT.Projs.Init **false** *(logical)*

Whether TBTRANS will only create the projection tables and then quit.

As TBTRANS allows to re-use the projection file the user can choose to stop right after creation. Specifically it will allow one to swap projection states with other projection states. This can be useful when bias is applied and the hybridisation "destroys" the molecule Hamiltonian. After initialising the projection tables the user can manually swap them with those calculated at zero bias, thus retaining the same projection tables for different bias'.

Note that for spin calculations you need to utilise the **TBT.Spin** flag to initialise both projection files (spin UP *and* spin DOWN) before proceeding with the calculation.

TBT.Projs.Debug **false** *(logical)*

Print out additional information regarding the projections. It will print out assertion lines orthogonality.

Possibly not useful for other than the developers.

%block TBT.Projs.T ⟨**None**⟩ *(block)*

depends on: **TBT.Projs**, **TBT.Projs.T.All**, **TBT.Projs.T.out**

As one might specify *many* molecular projections to investigate a lot of details of the system it seems perilous to always calculate all allowed transmission permutations.

Instead the user has to supply the permutations of transport that is calculated. This block will let the user decide which to calculate and which to not.

In the following **Left**(L)/**Right**(R) corresponds to $T = \mathrm{Tr}[\mathbf{G}\mathbf{\Gamma}_L\mathbf{G}^\dagger\mathbf{\Gamma}_R]$ where **Left**, **Right** are found in the **TBT.Elecs** block.

from <proj-L> to Projects $\mathbf{\Gamma}_L$ on to the **<projection>** before doing the R projections.
The R projections are constructed in the following lines until **end** is seen.

<proj-R> Projects $\mathbf{\Gamma}_R$ on to the **<projection>** which then calculates the transmission

Each projection can be represented in three different ways:

<elec> Makes no projection on the scattering matrix

<elec>.<name> Makes all permutations of the projections attached to the molecule named **<name>**

<elec>.<name>.<P-name> Projects the named projection **<P-name>** from molecule **<name>** onto electrode **<elec>**

An example input for projection two molecules could be:

```
%block TBT.Projs.T
  from Left.M-L.HOMO to
    Right.M-R
    Right
  end
  from Left.M-L.LUMO to
    Right.M-R.LUMO
```

```
  end
%endblock
```

which will be equivalent to the more verbose

```
%block TBT.Projs.T
  from Left.M-L.HOMO to
    Right.M-R.HOMO
    Right.M-R.LUMO
    Right.M-R.H-plus-L
    Right
  end
  from Left.M-L.LUMO to
    Right.M-R.LUMO
  end
%endblock
```

This will calculate the transport using all these equations

$$T_{|H_1\rangle,|H_2\rangle} = \mathrm{Tr}\left[\mathbf{G}|H_1\rangle\langle H_1|\mathbf{\Gamma}_L|H_1\rangle\langle H_1|\mathbf{G}^\dagger|H_2\rangle\langle H_2|\mathbf{\Gamma}_R|H_2\rangle\langle H_2|\right] \tag{18}$$

$$T_{|H_1\rangle,|L_2\rangle} = \mathrm{Tr}\left[\mathbf{G}|H_1\rangle\langle H_1|\mathbf{\Gamma}_L|H_1\rangle\langle H_1|\mathbf{G}^\dagger|L_2\rangle\langle L_2|\mathbf{\Gamma}_R|L_2\rangle\langle L_2|\right] \tag{19}$$

$$T_{|H_1\rangle,|H_2\rangle+|L_2\rangle} = \mathrm{Tr}\left[\mathbf{G}|H_1\rangle\langle H_1|\mathbf{\Gamma}_L|H_1\rangle\langle H_1|\mathbf{G}^\dagger\big(|H_2\rangle\langle H_2| + |L_2\rangle\langle L_2|\big)\mathbf{\Gamma}_R\big(|H_2\rangle\langle H_2| + |L_2\rangle\langle L_2|\big)\right] \tag{20}$$

$$T_{|H_1\rangle,R} = \mathrm{Tr}\left[\mathbf{G}|H_1\rangle\langle H_1|\mathbf{\Gamma}_L|H_1\rangle\langle H_1|\mathbf{G}^\dagger\mathbf{\Gamma}_R\right] \tag{21}$$

$$T_{|L_1\rangle,|L_2\rangle} = \mathrm{Tr}\left[\mathbf{G}|L_1\rangle\langle L_1|\mathbf{\Gamma}_L|L_1\rangle\langle L_1|\mathbf{G}^\dagger|L_2\rangle\langle L_2|\mathbf{\Gamma}_R|L_2\rangle\langle L_2|\right] \tag{22}$$

Notice that Eq. (20) is equivalent to (21) in our two state model.

Note that removing an explicit named projection allows easy creation of all available permutations of the projection states associated with the molecule.

By default some electrodes are not accessible for projections unless **TBT.Projs.T.Out** or **TBT.Projs.T.All** are **true**.

TBT.Projs.Only **false** *(logical)*

Whether TBtrans will not calculate non-projected transmissions. If you are only interested in the projection transmissions and/or have already calculated the non-projected transmissions you can use this option.

TBT.Projs.DOS.A **false** *(logical)*

depends on: **TBT.Atoms.Device**

Save the spectral density of states for the projections. In case you have set **TBT.DOS.A** this will default to that flag.

In case any of **TBT.Projs.Current.Orb**, **TBT.Projs.DM.A**, **TBT.Projs.COOP.A** or **TBT.Projs.COHP.A** is **true** this flag will be set to **true** as well.

TBT.Projs.Current.Orb **false** *(logical)*

depends on: **TBT.Atoms.Device**

Will calculate and save the orbital current for the device with the projections.

The orbital current will be saved in the same sparsity pattern as the cut-out device region sparsity pattern.

TBT.Projs.DM.A **false** *(logical)*

depends on: **TBT.Atoms.Device**

Calculate the energy and k-resolved density matrix for the projected spectral functions. The density matrix may be used to construct real-space LDOS profiles.

TBT.Projs.COOP.A **false** *(logical)*

depends on: **TBT.Atoms.Device**

Calculate COOP from the projected spectral function in the device region.

TBT.Projs.COHP.A **false** *(logical)*

depends on: **TBT.Atoms.Device**

Calculate COHP from the projected spectral function in the device region.

TBT.Projs.T.All **false** *(logical)*

Same as **TBT.T.All**, but for projections. If differing projections are performed on the scattering states the transmission will not be reversible. You can turn on all projection operations using this flag.

TBT.Projs.T.Out **false** *(logical)*

Same as **TBT.T.Out** for projections.

4.6.3 NetCDF4 support

TBtrans stores all relevant physical quantities in the `SystemLabel.TBT.nc` file which retains orbital resolved DOS, orbital currents, transmissions, transmission eigenvalues, etc. One may use sisl to easily analyze and extract quantities from this file using Python.

These files are created if NetCDF4 support is enabled

`SystemLabel.TBT.nc` File which contain nearly everything calculated in TBtrans. The structure of this file is a natural tree structure to accommodate N electrode output.

`SystemLabel.TBT.SE.nc` see **TBT.SelfEnergy.Save**

Down-folded self-energies are stored in this file.

`SystemLabel.TBT.Proj.nc` see **TBT.Projs**

Stores projected DOS, transmission and/or orbital currents. Using projections for large k and energy sampling will create very large files. Ensure that you have a large amount of disk-space available.

`SystemLabel.DOS` see **TBT.DOS.Gf**

The k resolved density of states from the Green function.

`SystemLabel.AVDOS` see **TBT.DOS.Gf**

The k averaged density of states from the Green function.

`SystemLabel.ADOS_<>` see **TBT.DOS.A**

The k resolved density of states from the electrode name <>.

`SystemLabel.AVADOS_<>` see **TBT.DOS.A**

The k averaged density of states from the electrode name <>.

`SystemLabel.TRANS_<1>_<2>`

The k resolved transmission from **<1>** to **<2>**.

`SystemLabel.AVTRANS_<1>_<2>`

The k averaged transmission from **<1>** to **<2>**.

`SystemLabel.CORR_<1>` see **TBT.T.Out**

The k resolved correction to the transmission for **<1>**.

`SystemLabel.AVCORR_<1>` see **TBT.T.Out**

The k averaged correction to the transmission for **<1>**.

`SystemLabel.TEIG_<1>_<2>` see **TBT.T.Eig**

The k resolved transmission eigenvalues from **<1>** to **<2>**.

`SystemLabel.AVTEIG_<1>_<2>` see **TBT.T.Eig**

The k averaged transmission eigenvalues from **<1>** to **<2>**.

`SystemLabel.CEIG_<1>` see **TBT.T.Out**

The k resolved correction eigenvalues for **<1>**.

`SystemLabel.AVCEIG_<1>` see **TBT.T.Out**

The k averaged correction eigenvalues for **<1>**.

`SystemLabel.BDOS_<>` see **TBT.DOS.Elecs/TBT.T.Bulk**

The k resolved bulk density of states of electrode <>.

`SystemLabel.AVBDOS_<>` see **TBT.DOS.Elecs/TBT.T.Bulk**

The k averaged bulk density of states of electrode <>.

`SystemLabel.BTRANS_<>` see **TBT.DOS.Elecs/TBT.T.Bulk**

The k resolved bulk transmission of electrode <>.

`SystemLabel.AVBTRANS_<>` see **TBT.DOS.Elecs/TBT.T.Bulk**

The k averaged bulk transmission of electrode <>.

All the above files will only be created if TBtrans was successfully executed and their respective options was enabled.

4.6.4 No NetCDF4 support

In case TBTRANS is not compiled with NetCDF4 support TBTRANS is heavily limited in functionality and subsequent analysis. Particularly the DOS quantities are not orbital resolved. Also none of the quantities will be k averaged, this is required to be done externally.

The following files are created:

`SystemLabel.DOS` see **TBT.DOS.Gf**

The k resolved density of states from the Green function.

`SystemLabel.ADOS_<>` see **TBT.DOS.A**

The k resolved density of states from the electrode name <>.

`SystemLabel.TRANS_<1>_<2>`

The k resolved transmission from **<1>** to **<2>**.

`SystemLabel.TEIG_<1>_<2>` see **TBT.T.Eig**

The k resolved transmission eigenvalues from **<1>** to **<2>**.

`SystemLabel.BDOS_<>` see **TBT.DOS.Elecs/TBT.T.Bulk**

The k resolved bulk density of states of electrode <>.

`SystemLabel.BTRANS_<>` see **TBT.DOS.Elecs/TBT.T.Bulk**

The k resolved bulk transmission of electrode <>.

References

[1] Nick Papior, Nicolás Lorente, Thomas Frederiksen, Alberto García, and Mads Brandbyge. Improvements on non-equilibrium and transport Green function techniques: The next-generation TranSiesta. *Computer Physics Communications*, 212:8–24, mar 2017. ISSN 00104655. doi: 10.1016/j.cpc.2016.09.022. URL `https://doi.org/10.1016/j.cpc.2016.09.022`.

[2] Nick Papior, Gaetano Calogero, Susanne Leitherer, and Mads Brandbyge. Removing all periodic boundary conditions: Efficient nonequilibrium Green's function calculations. *Physical Review B*, 100(19):195417, nov 2019. ISSN 2469-9950. doi: 10.1103/PhysRevB.100.195417. URL `https://link.aps.org/doi/10.1103/PhysRevB.100.195417`.

[3] Nick R. Papior. sisl, 2020. URL `https://doi.org/10.5281/zenodo.597181`.

Index

List of SIESTA files

List of fdf flags